SIMATS ENGINEERING

ASSESSMENT TOOL 2

REVERSE ENGINEERING TASK

COURSE CODE: CSA3713

COURSE NAME: SOFTWARE TESTING

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box and white-box testing techniques and control methods flow. (BL3)

Assessment Tool Weightage: 30%

Duration: 60 Minutes | Total Marks: 50 | Questions: 5

STUDENT DETAILS

Name: Sameer Ahmed G

Register Number: 192421154

CODE OF CONDUCT

I Sameer Ahmed G / 192421154 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sameer

Contents

- Q1. Reverse Engineering of a Black-Box Web Application Using Selenium
- Q2. Reverse Engineering of a Java Application Using JaCoCo
- Q3. Reverse Engineering and API Testing of a Banking Application
- Q4. Reverse Engineering a COTS Application
- Q5. Reverse Engineering a Python Application Using pytest and Coverage.py

Q1. Reverse Engineering of a Black-Box Web Application Using Selenium

Task title: Reverse Engineering and Test Case Design for a Web-Based Login System.

Objective: Reconstruct the observable functions of the university portal without relying on its unavailable original requirements or internal source code. The main functions are authentication, error validation, session creation, logout, and access control for protected pages.

a) Application reconnaissance and reverse engineering

Reverse engineering begins by treating the application as a black box: provide controlled inputs, observe visible outputs and navigation, and document the evidence. Chrome DevTools can expose DOM structure, element attributes, browser-console errors, network requests, response codes, cookies, and storage behavior. These observations help explain external behavior, but they do not prove how the hidden server-side code is implemented.

Functional model

```
Student -> Login Page -> Authentication / Validation -> Dashboard
                                    |                    |
                     Invalid input ---+                  +--> Attendance Page
                     Error message                        +--> Marks Page
                                                          +--> Course Registration
Dashboard / Protected Page -> Logout -> Logged Out
Logged Out -> Direct protected URL -> Redirect / Access Denied
```

Reconnaissance procedure

- Record the application URL, test account permissions, browser version, date, and build/version if visible.
- Inspect every reachable page, input field, button, link, navigation route, validation message, and empty-state message.
- Use the Elements panel to identify stable attributes such as id, name, aria-label, and data-testid. Prefer stable attributes over fragile absolute XPath selectors.
- Use the Network panel to observe request methods, URL paths, status codes, cookies, and redirects. Do not expose or publish real credentials, session cookies, or personal data.
- Check browser-console errors and record reproducible observations with screenshots or sanitized logs.
- Classify each note as Verified Observation, Assumption, or Question for the product owner. Do not infer a password policy, session timeout, or username-length limit unless evidence supports it.

Area	Verified from the supplied scenario	Must be confirmed by observation
Login	Username and password authentication; validation and error messages.	Required fields, exact messages, username/password rules, lockout rules.
Session	A session is created after successful login; logout terminates access.	Cookie/token mechanism, timeout period, session rotation, server-side invalidation.
Protected pages	Attendance, marks, and course registration are internal functions.	Exact URLs, role permissions, redirect/error behavior, direct-link protection.
Diagnostics	DevTools is available for inspection.	Observed endpoints, response codes, console errors, and network behavior.

A safe functional requirement derived from the supplied scenario is: "When valid credentials are submitted, the portal shall allow the student to access the dashboard and permitted internal pages." A separate negative requirement is: "When a user is logged out, direct access to a protected page shall not reveal protected content." Exact response text and undocumented constraints require confirmation.

b) Black-box test design

Equivalence Class Partitioning (ECP) divides inputs into groups expected to behave similarly. Boundary Value Analysis (BVA) tests values at and around documented limits. Since the question does not provide a username or password length limit, those limits must be discovered from approved specifications or the visible UI before numeric boundary cases are finalized.

Input	Valid equivalence class	Invalid equivalence classes	Expected behavior
Username	Existing, active, correctly formatted test username.	Empty; unknown username; prohibited format; over/under length only if defined.	Valid account continues authentication; invalid input is safely rejected.
Password	Correct password for the authorized test account.	Empty; incorrect; malformed/over-limit input where applicable.	Correct pair may authenticate; other inputs do not create an authenticated session.
Credential pair	Correct username and correct password.	Correct username/wrong password; unknown username/correct-looking password; empty fields.	Only valid credentials grant access; errors do not disclose sensitive account details.
Session	Active authenticated session.	No session; expired session; logged-out session.	Protected pages require a valid session.

Representative test-case table

ID / Preconditions	Input and steps	Expected result	Actual result / Status / Defect ID
WEB-01; logged out; authorized test account available.	Enter valid username and password; submit.	Dashboard opens; session is established; protected pages are accessible according to role.	Record after execution.
WEB-02; logged out.	Enter valid username and incorrect password; submit.	Login is rejected; no authenticated session is created; safe error is shown.	Record after execution.
WEB-03; logged out.	Leave username blank; enter password; submit.	Required-field validation appears or request is rejected; no session is created.	Record after execution.
WEB-04; logged out.	Enter username; leave password blank; submit.	Required-field validation appears or request is rejected; no session is created.	Record after execution.
WEB-05; logged in.	Select Logout, then revisit the login URL.	Session is terminated and login page is displayed.	Record after execution.
WEB-06; logged out.	Paste a known protected-page URL directly into the address bar.	Access is denied or redirected to login; protected data is not displayed.	Record after execution.
WEB-07; just logged out.	Use browser Back, then attempt refresh on a protected page.	After refresh / a protected request, access is denied. Check whether stale cached content can appear after logout and assess it against the security requirement.	Record after execution.
WEB-08; logged in, if timeout is supported.	Wait beyond documented session timeout and navigate to a protected page.	Session-expired behavior is shown and authentication is required again.	Record after execution.
WEB-09; constraints documented.	Test minimum length - 1, minimum, minimum + 1 and equivalent maximum values.	Inputs are handled according to documented boundary rules; no crash or bypass.	Record after execution.
WEB-10; logged out.	Try a long string, punctuation, Unicode, and common injection-like strings in fields.	Input is safely handled; no crash, script execution, authentication bypass, or sensitive stack trace.	Record after execution.

Random testing supplements the planned tests by generating additional username/password combinations, lengths, character sets, and navigation sequences. Use a reproducible random seed and a controlled test account. Random inputs can expose unanticipated parser errors or unexpected transitions, but they do not replace requirement-based test cases.

c) Automation using Selenium

The following Python Selenium example is a template. Replace BASE_URL and the locators with values verified in DevTools. Run only against an authorized test instance. A stable locator such as id or data-testid is preferred. The example captures a screenshot if a test fails.

```
# pip install selenium pytest
import os
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
```

```
from selenium.webdriver.support import expected_conditions as EC

BASE_URL = os.environ.get("PORTAL_URL", "http://localhost:8000")
TEST_USER = os.environ.get("PORTAL_TEST_USER", "replace-test-user")
TEST_PASS = os.environ.get("PORTAL_TEST_PASS", "replace-test-password")

@pytest.fixture
def driver():
    options = webdriver.ChromeOptions()
    # Set headless mode only when required by the test environment.
    browser = webdriver.Chrome(options=options)
    browser.set_window_size(1365, 900)
    yield browser
    browser.quit()

def wait_for(driver, locator):
    return WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located(locator)
    )

def test_valid_login_and_logout(driver):
    driver.get(f"{BASE_URL}/login")
    wait_for(driver, (By.ID, "username")).send_keys(TEST_USER)
    driver.find_element(By.ID, "password").send_keys(TEST_PASS)
    driver.find_element(By.ID, "login-button").click()
    WebDriverWait(driver, 10).until(EC.url_contains("dashboard"))
    assert "dashboard" in driver.current_url.lower()
    wait_for(driver, (By.ID, "logout-button")).click()
    WebDriverWait(driver, 10).until(EC.url_contains("login"))

def test_invalid_login(driver):
    driver.get(f"{BASE_URL}/login")
    wait_for(driver, (By.ID, "username")).send_keys(TEST_USER)
    driver.find_element(By.ID, "password").send_keys("definitely-wrong-password")
    driver.find_element(By.ID, "login-button").click()
    error = wait_for(driver, (By.ID, "login-error"))
    assert error.is_displayed()
    assert "dashboard" not in driver.current_url.lower()

def test_protected_page_requires_login(driver):
    driver.get(f"{BASE_URL}/attendance")
    WebDriverWait(driver, 10).until(
        lambda d: "login" in d.current_url.lower()
        or "access denied" in d.page_source.lower()
    )
    assert "attendance data" not in driver.page_source.lower()

# pytest --html=report.html (install pytest-html if HTML output is needed)
```

Selenium 4 manages many browser drivers automatically when compatible browser versions are available. In controlled environments, pin the browser and driver versions. Never hard-code real passwords in source control; use environment variables or a secure CI secret store. Test selectors and assertions must match the actual application.

d) State and session testing

```
Logged Out --submit credentials--> Login in Progress
Login in Progress --valid credentials--> Logged In
Login in Progress --invalid credentials--> Logged Out (error shown)
Logged In --logout--> Logged Out
Logged In --timeout / session invalidation--> Session Expired
Session Expired --login again--> Login in Progress
Logged Out / Session Expired --direct protected URL--> Denied or redirected to Login
```

Transition	Test stimulus	Expected result
Logged Out -> Login in Progress	Submit credentials.	Authentication attempt starts; duplicate submissions are handled safely.
Login in Progress -> Logged In	Provide valid credentials.	Dashboard appears; session is active.
Login in Progress -> Logged Out	Provide invalid credentials.	No authenticated state; clear safe error appears.
Logged In -> Logged Out	Select logout.	Session invalidated and protected requests rejected.
Logged In -> Session Expired	Wait past configured timeout.	Protected operation requires re-authentication.
Logged Out -> protected page	Enter internal URL directly or use Back.	No unauthorized content becomes accessible.

Important invalid transitions include reaching an internal page without authentication, keeping a working session after logout, using an expired session to retrieve protected data, and obtaining an authenticated state after invalid credentials.

e) Test adequacy evaluation

Test adequacy should be assessed by traceability and risk, not by the number of tests alone. For every functional requirement, identify at least one positive test, negative test, and applicable boundary or state-transition test. High-risk security requirements such as logout invalidation and protected-page access receive higher priority.

```
Execution coverage (%) = Executed planned test cases / Total planned test cases x 100
Pass rate among executed (%) = Passed test cases / Executed test cases x 100
Overall planned pass proportion (%) = Passed test cases / Total planned test cases x
100
```

Illustrative calculation only: if 10 test cases are planned, 8 are executed, and 7 pass, execution coverage is 8/10 x 100 = 80%; pass rate among executed tests is 7/8 x 100 = 87.5%; overall planned pass proportion is 7/10 x 100 = 70%. Actual figures must be calculated from the execution report.

Release readiness requires all critical functions to be tested, no open unacceptable-severity defects, protected-page authorization to be verified, regression tests to pass, and the remaining risks to be explicitly accepted by the responsible stakeholder. A high pass percentage alone is not enough if a critical requirement remains untested.

Expected deliverables checklist

- Functional model and observation log
- Equivalence classes, boundary analysis, and structured test-case spreadsheet
- Selenium scripts and failure screenshots from actual test execution
- State-transition model and defect report

• Test adequacy calculations and release-readiness recommendation

Q2. Reverse Engineering of an Existing Java Application Using JaCoCo

Task title: White-Box Reverse Engineering and Code Coverage Analysis of a Student Result Processing Application.

The given method accepts an integer mark and returns Invalid, A, B, C, or Fail. Source code is available, so white-box analysis can identify decision structure and paths, while black-box techniques derive tests from the implied grading thresholds.

a) Reverse engineering and logic reconstruction

Recovered functional rules

• If mark is below 0 or above 100, return Invalid.

• For a valid mark of 90 through 100, return A.

• For a valid mark of 75 through 89, return B.

• For a valid mark of 50 through 74, return C.

• For a valid mark of 0 through 49, return Fail.

Flowchart

```
Start -> Read mark -> mark < 0 OR mark > 100? --Yes--> Invalid -> End
                                    | No
                                mark >= 90? --Yes--> A -> End
                                    | No
                                mark >= 75? --Yes--> B -> End
                                    | No
                                mark >= 50? --Yes--> C -> End
                                    | No
                                  Fail -> End
```

Control Flow Graph (CFG) nodes and edges

Node	Operation / decision	Outgoing edges
N1	Entry and read mark.	N2
N2	mark < 0?	True -> N4 (Invalid); False -> N3
N3	mark > 100?	True -> N4 (Invalid); False -> N5
N4	Return Invalid. Shared output node for both invalid branches.	N12
N5	mark >= 90?	True -> N6 (A); False -> N7
N6	Return A.	N12
N7	mark >= 75?	True -> N8 (B); False -> N9
N8	Return B.	N12
N9	mark >= 50?	True -> N10 (C); False -> N11 (Fail)
N10	Return C.	N12
N11	Return Fail.	N12
N12	Exit.	-

The compound invalid-input check can be represented as two predicate nodes when short-circuit evaluation is modelled explicitly. This distinction matters when calculating cyclomatic complexity: the number of independent Boolean predicates included in the CFG should match the graph used in the calculation.

Assumptions requiring confirmation: marks are integers; 0 and 100 are valid; grade boundaries are inclusive as written; the program returns a string; and there is no separate attendance rule in this Java method. These are inferred from code and should be checked against approved academic policy.

b) Black-box test design

Test ID	Mark input	Equivalence / boundary purpose	Expected output
JAVA-01	-1	Just below valid range.	Invalid
JAVA-02	0	Minimum valid mark.	Fail
JAVA-03	49	Just below pass threshold.	Fail
JAVA-04	50	Exact C threshold.	C
JAVA-05	74	Just below B threshold.	C
JAVA-06	75	Exact B threshold.	B
JAVA-07	89	Just below A threshold.	B
JAVA-08	90	Exact A threshold.	A
JAVA-09	100	Maximum valid mark.	A
JAVA-10	101	Just above valid range.	Invalid

ECP classes are: mark < 0 (invalid), 0-49 (Fail), 50-74 (C), 75-89 (B), 90-100 (A), and mark > 100 (invalid). BVA is especially effective at -1/0, 49/50, 74/75, 89/90, and 100/101 because off-by-one errors frequently occur at branch thresholds. Random tests can add reproducible values within and outside these ranges, but threshold-focused tests remain essential.

c) White-box test design and cyclomatic complexity

With short-circuit logic represented explicitly, the graph has five predicate nodes: mark < 0, mark > 100, mark >= 90, mark >= 75, and mark >= 50. The full CFG above contains N = 12 nodes and E = 16 edges, with P = 1 connected component. Therefore, V(G) = E - N + 2P = 16 - 12 + 2(1) = 6. This equals the five predicate nodes plus one. The graph and calculation use the same convention: each side of the compound OR is represented as a separate short-circuit decision.

A basis set of independent paths is shown below. The first two paths cover the two outcomes of the compound out-of-range check; the remaining paths cover each grading outcome.

Path	Representative condition	Test input	Output
P1	mark < 0 is true.	-1	Invalid
P2	mark < 0 is false, then mark > 100 is true.	101	Invalid
P3	Valid mark; mark >= 90 is true.	90	A
P4	Below 90; mark >= 75 is true.	75	B
P5	Below 75; mark >= 50 is true.	50	C
P6	All grade thresholds false.	49	Fail

JUnit 5 test example

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class ResultCalculatorTest {
    @Test void rejectsValuesBelowZero() {
        assertEquals("Invalid", ResultCalculator.calculateGrade(-1));
    }
    @Test void acceptsMinimumAndFails() {
        assertEquals("Fail", ResultCalculator.calculateGrade(0));
    }
    @Test void checksThresholds() {
        assertEquals("Fail", ResultCalculator.calculateGrade(49));
        assertEquals("C", ResultCalculator.calculateGrade(50));
        assertEquals("C", ResultCalculator.calculateGrade(74));
        assertEquals("B", ResultCalculator.calculateGrade(75));
        assertEquals("B", ResultCalculator.calculateGrade(89));
        assertEquals("A", ResultCalculator.calculateGrade(90));
        assertEquals("A", ResultCalculator.calculateGrade(100));
    }
    @Test void rejectsValuesAboveHundred() {
        assertEquals("Invalid", ResultCalculator.calculateGrade(101));
    }
}
```

Statement coverage measures whether each executable statement ran. Branch coverage measures whether each decision outcome (true and false) ran. Condition coverage measures whether each atomic Boolean condition evaluated both true and false. Because the first decision uses an OR, tests -1 and 101 help exercise each atomic condition, while 0 and 100 help show the out-of-range check is false overall. A suite that executes every return statement may still miss a decision outcome or a boundary error.

d) Tool-based JaCoCo coverage analysis

JaCoCo instruments Java bytecode and generates coverage information for the test run. It can report instruction, line, branch, method, class, and complexity coverage. Coverage numbers are evidence of execution, not proof that the grade rules are correct.

Maven configuration example

```
<!-- Add the following plugin inside <build><plugins> in pom.xml -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>

<!-- Run the suite and report -->
mvn clean test
<!-- Report typically appears under target/site/jacoco/index.html -->
```

Use a JaCoCo version compatible with the project JDK and Maven setup. After running tests, open the HTML report, inspect uncovered lines and red/yellow branch markers, add tests for missing outcomes, and regenerate the report. Record the observed report values rather than inserting assumed percentages.

Coverage item	What to inspect	Action if incomplete
Instruction / line	Did executable statements run?	Add tests for uncovered return paths or validations.
Branch	Were true and false outcomes of each decision exercised?	Add threshold and invalid-range test inputs.
Method / class	Was the calculator method executed?	Ensure the test suite actually calls the production method.
Complexity / missed paths	Which independent paths are not covered?	Create representative cases for every grade result and both invalid boundaries.

e) Test adequacy evaluation

Compare the initial and improved test suites using the same build and test environment. Record the number of tests, failures, instruction coverage, branch coverage, uncovered lines, and defects found in a before-and-after table. If no initial results exist, mark the baseline as “not measured” rather than inventing data.

Measure	Initial suite	Improved suite	Interpretation
Executed tests	To be measured	To be measured	Confirms that tests ran against the current build.
Instruction / line coverage	To be measured	To be measured	Shows the amount of code executed.
Branch coverage	To be measured	To be measured	Shows decision outcomes exercised.

Measure	Initial suite	Improved suite	Interpretation
Threshold coverage	Check existing cases	Include all threshold-adjacent values	Detects off-by-one errors.
Defects identified	Record actual count	Record actual count	Shows observed findings, not all possible defects.

High coverage does not establish that all grading requirements are correct. Requirement correctness needs traceability to approved rules, boundary tests, invalid-input tests, code review, and potentially mutation testing. For example, changing mark >= 90 to mark > 90 should be detected by a test at 90; changing mark >= 50 to mark > 50 should be detected by a test at 50. Release should be recommended only after the required rules are verified and unacceptable defects are resolved or formally accepted.

Q3. Reverse Engineering and API Testing of a Banking Application

Task title: Reverse Engineering of a Banking REST API and Validation of Transaction Rules.

Safety boundary: Use only an authorized training API, a mock service, or a locally hosted sample. Do not send tests to a real banking service without written authorization. The exact endpoint names, request fields, and status codes are not supplied, so the examples below are an illustrative contract to be mapped to the authorized API.

a) API reverse engineering

API reverse engineering records externally visible requests and responses to infer functionality. In Postman, inspect available documentation, collection files, environment variables, HTTP method, path, headers, body, status code, response schema, timing, and error format. Record both what is observed and what remains unconfirmed.

Illustrative API inventory - confirm before execution

Purpose	Example endpoint / method	Typical input	Expected behavior to verify
Authentication	POST /auth/login	Test credentials or training token.	Issue a valid test token or reject invalid credentials.
Balance retrieval	GET /accounts/{accountId}/balance	Authorized account identifier and token.	Return the authorized account balance without exposing another account.
Transfer submission	POST /transfers	Beneficiary, amount, and required transaction metadata.	Validate amount, balance, daily limit, authentication, and transaction state.
Transaction status	GET /transfers/{transferId}	Known transfer ID and authorized token.	Return a consistent transaction status where supported.

API interaction diagram

```
Client -> Authenticate -> Receive test token
Client -> GET account balance (authorized token) -> Balance response
Client -> POST transfer (token + beneficiary + amount) -> Validation
Validation -> Pending -> Successful OR Failed / Cancelled (if supported)
Client -> Query transfer status -> Confirm final state and balance consistency
```

Use the Network panel or Postman console to capture sanitized request-response pairs. Store secrets in Postman variables, not in screenshots or shared collections. Avoid using real account data. If the schema, authentication method, or status codes cannot be observed, record them as unknown instead of guessing.

b) Black-box test design

Apply ECP to transaction amounts: non-numeric/missing; zero or negative; valid positive amount within balance and daily limit; amount greater than balance; amount exceeding daily limit; and amount at the documented maximum. Use BVA around the minimum positive amount, available balance, and daily limit. Define exact numeric boundaries from the authorized API contract.

ID	Scenario / data	Expected result (confirm status contract)
API-01	Authenticated request; amount is a valid positive value within both balance and daily limit.	Transfer accepted into the defined workflow; exactly one transaction record is created.
API-02	amount = 0.	Rejected as invalid; balance remains unchanged.
API-03	amount < 0.	Rejected as invalid; balance remains unchanged.
API-04	amount is just below the available balance.	Accepted if all other rules are valid; balance updated correctly after successful completion.
API-05	amount equals available balance.	Outcome matches the documented rule; no overdraft occurs.
API-06	amount exceeds available balance by the smallest permitted unit.	Rejected for insufficient funds; no debit occurs.
API-07	amount equals the remaining daily limit.	Accepted if policy allows exact-limit transactions.
API-08	amount exceeds the daily limit by the smallest permitted unit.	Rejected; balance remains unchanged.
API-09	Missing amount, malformed JSON, string instead of number, or null.	Clear validation error; no unintended transaction.
API-10	No authorization token / invalid token.	Unauthorized request cannot complete a transfer.
API-11	Invalid account or beneficiary identifier.	Request rejected without changing balances.
API-12	Unsupported HTTP method or unexpected content type.	Request rejected safely according to the API contract.

Decision table for the core transfer rules

Rule / condition	T1	T2	T3	T4
Authenticated?	Yes	Yes	Yes	No
Amount > 0?	Yes	No	Yes	Yes
Balance sufficient?	Yes	N/A	No	N/A
Daily limit satisfied?	Yes	N/A	N/A	N/A
Expected action	Accept / process	Reject amount	Reject insufficient balance	Reject unauthorized request

Random testing can create reproducible positive amounts within the allowed range and malformed values such as strings, decimals beyond supported precision, missing fields, nulls, very large values, and unexpected JSON shapes. Use a fixed seed and a mock account so that repeated runs can be compared. For transactional APIs, verify not only the response body but also the resulting balance and transaction record.

c) State transition and transaction testing

```
Pending -> Successful (gateway confirms completion)
Pending -> Failed (declined, timeout, or validated failure)
Pending -> Cancelled (only if cancellation is supported and allowed)
Failed / Cancelled -> Pending should occur only through an explicitly supported retry
flow
Repeated submission -> same transaction result or safely rejected; never an unintended
duplicate debit
```

Test normal and abnormal transitions. A timeout does not automatically mean that the transfer failed on the server; query the transaction status before retrying. Where the API supports an idempotency key, submit the same key twice and verify that only one transfer is created. Where no such capability exists, document the limitation and test duplicate-submission handling using the approved contract.

d) White-box and control-flow analysis

If transaction source code is supplied, create a CFG for authentication, amount validation, balance checking, daily-limit checking, transaction persistence, and response handling. Count nodes and edges in that graph and calculate V(G) = E - N + 2P. Identify basis paths and test true and false outcomes of each decision.

If source code is not available, internal CFG, statement coverage, branch coverage, and cyclomatic complexity cannot be reliably measured from black-box observations. Do not estimate these values from the API behavior. Use observable evidence instead: requirements coverage, equivalence-class coverage, boundary coverage, decision-table coverage, transition coverage, response validation, and transaction-state consistency.

e) Test adequacy and defect reporting

Requirement	Test IDs	Evidence required	Status
Amount must be > 0	API-02, API-03	Request, response, and before/after balance.	To be executed
Amount <= available balance	API-04 to API-06	Balance snapshot and transaction response.	To be executed
Daily limit enforced	API-07, API-08	Remaining limit and response.	To be executed
Authentication required	API-10	Sanitized request and response.	To be executed
Failure does not debit balance	API-02, API-03, API-06, API-08, failure transition tests	Balance before/after and transaction status.	To be executed
Repeated request does not create duplicate transfer	Duplicate/idempotency scenario	Transaction IDs and balance change.	To be executed

Defect report minimum fields

Defect ID; title; environment and build; endpoint and HTTP method; sanitized headers and request body; preconditions; steps to reproduce; expected result; actual result; status code; severity; priority; timestamps; transaction IDs; balance before and after; logs; and owner/status. Do not include real credentials or full sensitive account information.

Measure requirement coverage as requirements with at least one executed test divided by total approved requirements. Measure decision-table coverage by the rules exercised. Include code coverage only when source code and instrumentation are available. A release decision requires critical business rules to pass, failures to leave data consistent, security controls to work, and any remaining high risks to be accepted explicitly.

Q4. Reverse Engineering a Commercial Off-The-Shelf (COTS) Application

Task title: Functional, Compatibility, and Performance Testing of an Existing COTS Learning Management System (LMS).

Because source code is unavailable, testing focuses on observable functionality, approved requirements, integration points, compatibility, usability, security behavior, and measured performance. Internal path coverage and JaCoCo-style instrumentation are not available unless the vendor provides an instrumentable build or suitable telemetry.

a) Reverse engineering of observable functionality

Functional decomposition

```
COTS Learning Management System
  |-- Authentication and Session Management
  |-- Course Catalog and Enrollment
  |-- Assignment Upload and Submission Status
  |-- Result / Grade Viewing
```

```
|-- Notifications and Error Messages
|-- Browser / Device Presentation
|-- Administration and Vendor Integration (where included)
```

Explore the UI using valid and invalid inputs, inspect page navigation and visible validations, review vendor documentation and release notes, and compare the observed behavior with approved operational requirements. Vendor clarification is needed for ambiguous file limits, accepted formats, late-submission rules, enrollment limits, supported browsers, accessibility requirements, and performance service-level agreements (SLAs).

b) Black-box test design

Feature	Valid classes	Invalid classes / boundaries	Test focus
Login fields	Active permitted user and valid credentials.	Empty fields, wrong password, disabled account, boundary lengths per specification.	Authentication, validation, safe errors, session handling.
Assignment file size	File below and at permitted maximum, if allowed.	Zero-byte file if prohibited; size just above maximum; corrupted file.	Accepted/rejected result, no partial submission.
File type	Vendor-approved formats.	Unsupported extension, mismatched extension/MIME type, malformed file.	Server-side validation, safe rejection, no execution of active content.
Enrollment	Eligible user and open course with available capacity.	Closed course, duplicate enrollment, ineligible user, full class.	Decision rules, duplicate prevention, clear response.
Submission timing	Submission before deadline.	At deadline and after deadline; timezone boundary.	Draft/Submitted/Late/Rejected behavior per policy.

Decision table for assignment submission (rules must be confirmed)

Condition / rule	T1	T2	T3	T4
User authenticated and authorized?	Yes	Yes	Yes	No
File format allowed?	Yes	No	Yes	N/A
File size within limit?	Yes	N/A	No	N/A
Submitted before deadline?	Yes	N/A	N/A	N/A
Expected outcome	Submitted / accepted	Rejected with validation	Rejected with size message	Access denied / login required

Pairwise compatibility matrix

Assume that Windows/macOS, Chrome/Edge, and Laptop/Mobile are all vendor-supported for this example. Replace these factors with the university's real supported configurations. Four carefully selected combinations cover every pair of values across the three binary factors.

Test	Browser	Operating system	Device	Pairs covered by this row
C1	Chrome	Windows	Laptop	Chrome-Windows; Chrome-Laptop; Windows-Laptop
C2	Chrome	macOS	Mobile	Chrome-macOS; Chrome-Mobile; macOS-Mobile
C3	Edge	Windows	Mobile	Edge-Windows; Edge-Mobile; Windows-Mobile
C4	Edge	macOS	Laptop	Edge-macOS; Edge-Laptop; macOS-Laptop

State transition examples for assignment submission are Draft -> Submitted (valid submission), Draft -> Rejected (invalid file or unauthorized user), Submitted -> Late only if the system permits post-deadline status changes, and Submitted -> Accepted/Graded when the instructor processes the work. Exact states and transitions must follow LMS behavior and policy.

c) Tool-based functional and performance testing

Selenium workflow

- Create a dedicated test course, user, and assignment with known deadline and file-size/type limits.
- Automate login, course opening, assignment upload, submission confirmation, result viewing, and logout.
- Use explicit waits for visible elements and state changes; avoid fixed sleep calls where possible.
- Check the success message and resulting submission status; capture screenshots on failure.
- Keep selectors configurable because actual LMS markup was not supplied.

```
# Selenium template: replace URL and locators after inspecting the LMS.
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

base_url = "http://localhost:8080"  # authorized test instance only
browser = webdriver.Chrome()
try:
    wait = WebDriverWait(browser, 10)
    browser.get(base_url + "/login")
    wait.until(EC.visibility_of_element_located((By.ID, "username"))).send_keys("test-student")
    browser.find_element(By.ID, "password").send_keys("set-through-secure-test-config")
    browser.find_element(By.ID, "login-button").click()
    wait.until(EC.visibility_of_element_located((By.ID, "dashboard")))
    # Continue with course, file-input, submit-button, and status assertions
    # only after their actual locators are confirmed in DevTools.
finally:
    browser.quit()
```

Apache JMeter performance plan

- Obtain written authorization and select a non-production environment with test data.
- Define a workload model: expected concurrent users, ramp-up duration, test duration, and key user journeys. Derive these from the SLA or stakeholder requirements; do not invent an acceptance threshold.
- In JMeter, create a Test Plan and Thread Group. Add HTTP Request Defaults, HTTP Header Manager, Cookie Manager or token handling as required, and requests for approved endpoints.
- Add assertions for expected HTTP codes and response content. Add timers that approximate realistic user behavior and avoid unintentionally overwhelming the service.
- Run a small smoke load first, then the approved load, stress, and endurance profiles. Capture response-time percentiles, throughput, error rate, and resource data if available.
- Compare measurements against predefined criteria and export the JMeter results/report. Do not run destructive tests or load production without explicit authorization.

Metric	Meaning	Report field
Response time	Elapsed time for a request or transaction.	Median, 90th/95th percentile, and maximum.
Throughput	Requests or completed business transactions per unit time.	Requests/sec or transactions/minute.
Error rate	Share of failed requests or assertions.	Failed samples / total samples x 100.
Load level	Concurrent users and workload duration.	Thread count, ramp-up, duration, scenario mix.
Acceptance criteria	Thresholds defined by requirements/SLA.	Pass/fail against approved threshold; not guessed.

d) COTS testing limitations

Without source code, testers cannot construct a trustworthy internal CFG, directly enumerate internal execution paths, measure statement/branch coverage, or apply JaCoCo-style bytecode instrumentation. The vendor may also restrict database access, logs, debug modes, and internal configuration. These limits reduce visibility into internal causes but do not prevent thorough external acceptance testing.

Alternative evidence includes approved requirements, vendor documentation, release notes, visible UI behavior, API/interface contracts, integration testing, browser DevTools, network logs, security test results authorized by the vendor, performance measurements, support commitments, known limitations, and acceptance test results. Defects should be described through reproducible input-output behavior, not unsupported claims about the hidden implementation.

e) Test adequacy evaluation

Use a weighted acceptance matrix to make release decisions transparent. The weights below are a suggested starting point only; the university must approve or adjust them based on risk and operational priorities.

Quality area	Suggested weight	Evidence	Score (0-100)
Functionality	30%	Requirement traceability, positive/negative cases, assignment/enrollment workflows.	To be measured
Compatibility	20%	Pairwise browser, operating-system, and device results.	To be measured
Security	20%	Authentication, authorization, session, upload validation, and vendor security evidence.	To be measured
Performance	20%	JMeter load results compared with approved SLA.	To be measured
Usability	10%	Navigation, error clarity, accessibility and user feedback.	To be measured

Weighted score = sum of (area weight x area score). For example, if the measured functionality score is F, its contribution to the total is 0.30 x F. Do not calculate a final score until real evidence exists. A high total must not override a critical security failure or an unmet mandatory requirement.

Recommend Acceptance only when mandatory criteria pass and no unacceptable critical defects remain. Recommend Conditional Acceptance when the remaining risks are non-critical, documented, time-bound, and formally accepted with a mitigation plan. Recommend Rejection when critical requirements fail, high-risk defects remain unresolved, or the evidence is insufficient to demonstrate safe operation.

Q5. Reverse Engineering a Python Application Using pytest and Coverage.py

Task title: Reverse Engineering, Path Testing, and Test Adequacy Evaluation of a Result Validation Program.

The given Python function validates marks and attendance, then returns Invalid Mark, Invalid Attendance, Attendance Shortage, Pass, or Fail. The order of conditions is important because the function returns as soon as the first matching rule is encountered.

a) Reverse engineering and specification recovery

Recovered behavior from the supplied code

- If mark < 0 or mark > 100, return Invalid Mark.
- Otherwise, if attendance < 0 or attendance > 100, return Invalid Attendance.

- Otherwise, if attendance < 75, return Attendance Shortage.
- Otherwise, if mark >= 50, return Pass.
- Otherwise, return Fail.

Flowchart

```
Start -> Read mark, attendance -> mark < 0 OR mark > 100?
  Yes -> Invalid Mark -> End
  No -> attendance < 0 OR attendance > 100?
       Yes -> Invalid Attendance -> End
       No -> attendance < 75?
            Yes -> Attendance Shortage -> End
            No -> mark >= 50?
                 Yes -> Pass -> End
                 No -> Fail -> End
```

The function implies inclusive valid ranges of 0-100 for both inputs, an attendance eligibility threshold of 75%, and a pass mark of 50. The function checks the mark before attendance; if both are invalid, Invalid Mark is returned first. Stakeholders must confirm that marks and attendance should be numeric, whether non-integer values are allowed, whether exactly 75% qualifies, how missing values and wrong data types should be handled, and whether attendance checks take precedence over marks.

b) Black-box test design

ID	mark	attendance	Purpose	Expected output
PY-01	-1	80	Mark just below minimum.	Invalid Mark
PY-02	0	75	Minimum valid mark; eligible attendance.	Fail
PY-03	49	75	Just below pass mark.	Fail
PY-04	50	75	Exact pass threshold.	Pass
PY-05	100	75	Maximum valid mark.	Pass
PY-06	101	75	Mark above maximum.	Invalid Mark
PY-07	60	-1	Attendance below minimum.	Invalid Attendance
PY-08	60	101	Attendance above maximum.	Invalid Attendance
PY-09	80	74	Attendance just below eligibility threshold.	Attendance Shortage
PY-10	80	75	Attendance at threshold.	Pass
PY-11	49	100	Valid range but mark is below 50.	Fail
PY-12	-1	101	Both inputs invalid; confirms decision order.	Invalid Mark

ECP partitions marks into below 0, 0-49, 50-100, and above 100 (the 50-100 range can be subdivided by grade/pass rules if needed). Attendance partitions into below 0, 0-74, 75-100, and above 100. BVA targets -1/0/1, 49/50/51, 74/75/76, and 100/101. Error guessing should include None, strings, floating-point values, extremely large integers, missing arguments, and simultaneous invalid inputs. The current function may raise TypeError for incompatible data types; this should be tested and handled if required by the specification.

Decision table for the main behavior

Condition / outcome	**R1**	**R2**	**R3**	**R4**	**R5**
Mark out of range?	Yes	No	No	No	No
Attendance out of range?	N/A	Yes	No	No	No
Attendance < 75?	N/A	N/A	Yes	No	No
Mark >= 50?	N/A	N/A	N/A	Yes	No
Expected output	Invalid Mark	Invalid Attendance	Attendance Shortage	Pass	Fail

c) White-box test design and automation

There are four if-statements and two logical OR operators. If the CFG represents each short-circuit atomic condition separately, its decision predicates are mark < 0, mark > 100, attendance < 0, attendance > 100, attendance < 75, and mark >= 50: six predicates, giving V(G) = 6 + 1 = 7. In the matching explicit CFG, there are N = 13 nodes and E = 18 edges with P = 1 connected component, so V(G) = E - N + 2P = 18 - 13 + 2(1) = 7. Both methods agree.

Independent path	**Condition sequence**	**Representative input**	**Expected output**
P1	mark < 0 is true.	mark=-1, attendance=80	Invalid Mark
P2	mark < 0 false; mark > 100 true.	mark=101, attendance=80	Invalid Mark
P3	Valid mark; attendance < 0 true.	mark=80, attendance=-1	Invalid Attendance
P4	Valid mark; attendance < 0 false; attendance > 100 true.	mark=80, attendance=101	Invalid Attendance
P5	Both values valid; attendance < 75 true.	mark=80, attendance=74	Attendance Shortage
P6	Attendance eligible; mark >= 50 true.	mark=50, attendance=75	Pass
P7	Attendance eligible; mark >= 50 false.	mark=49, attendance=75	Fail

pytest example

```
# Save production function in result_validator.py
# Save the following tests in test_result_validator.py
import pytest
from result_validator import validate_result

@pytest.mark.parametrize("mark, attendance, expected", [
    (-1, 80, "Invalid Mark"),
    (101, 80, "Invalid Mark"),
    (0, 75, "Fail"),
    (49, 75, "Fail"),
    (50, 75, "Pass"),
    (100, 75, "Pass"),
    (80, -1, "Invalid Attendance"),
    (80, 101, "Invalid Attendance"),
    (80, 74, "Attendance Shortage"),
    (80, 75, "Pass"),
    (-1, 101, "Invalid Mark"),
])
def test_validate_result(mark, attendance, expected):
    assert validate_result(mark, attendance) == expected

# Run: pytest -q
```

Statement coverage ensures all return statements execute. Branch coverage tests true and false outcomes of each decision. Condition coverage checks each atomic Boolean condition in both true and false states. Path testing targets the independent routes listed above. With compound OR expressions, the input values should intentionally activate each side of the OR, not only the overall decision result.

d) Additional white-box testing

Data-flow testing follows where variables are defined and used. In this function, mark and attendance are function inputs, defined at entry, and are used by range checks, the threshold comparison, and the pass/fail decision. Tests for lower and upper bounds show whether each input reaches the relevant checks; tests where both inputs are invalid confirm the order in which definitions are validated.

Mutation testing examples

Mutation	Test that should detect it	Reason
Change attendance < 75 to attendance <= 75.	mark=80, attendance=75 should return Pass in the original code.	Mutant incorrectly returns Attendance Shortage at exactly 75.
Change mark >= 50 to mark > 50.	mark=50, attendance=75.	Mutant incorrectly returns Fail at the pass boundary.
Remove mark > 100 validation.	mark=101, attendance=80.	Mutant may return a grade for an invalid mark.
Remove attendance > 100 validation.	mark=80, attendance=101.	Mutant may process an invalid attendance value.
Swap the order of mark and attendance validation.	mark=-1, attendance=101.	Expected priority may change; this test exposes the changed behavior if validation order is a requirement.

A mutation is "killed" when at least one test fails against the mutated implementation but passes against the original. A surviving mutant may indicate a missing test, an equivalent mutation, or a requirement that does not distinguish the behaviors. Mutation testing evaluates the fault-detection ability of the suite rather than only the percentage of executed code.

e) Evaluate test adequacy

Coverage.py commands

```
# Install test and coverage tools
python -m pip install pytest coverage

# Measure statement and branch coverage
coverage run --branch -m pytest
coverage report -m
coverage html

# Open htmlcov/index.html to inspect missing lines and branches
```

Compare initial and improved test suites using statement coverage, branch coverage, boundary coverage, and requirements traceability. The initial result and improved result must be measured by running the test commands in the actual project; values cannot be inferred merely from the number of test cases.

Measure	How to assess adequacy	Evidence to record
Statement coverage	Every executable statement / return path is executed.	Coverage.py report and uncovered lines.
Branch coverage	Both true and false sides of each decision are exercised.	Branch report and tests tied to each branch.
Boundary coverage	Values just below, at, and above each important threshold are tested.	Mark/attendance boundary matrix.
Requirement traceability	Each approved rule maps to one or more tests.	Requirement-to-test matrix with status.
Mutation testing	Important mutants are detected by at least one test.	Mutants attempted, killed, survived, and rationale.

High coverage alone cannot prove correctness: a test may execute a line without asserting the right result, and a test suite may encode the same wrong assumption as the production code. Release requires stakeholder-approved rules, all high-risk branches and boundaries exercised, invalid types handled according to the specification, no unacceptable open defects, and review of mutation survivors. The final report should state what was measured and what remains unverified.

OVERALL CONCLUSION

Reverse engineering enables testers to recover an evidence-based understanding of an existing system when original documentation or test cases are missing. Black-box methods such as equivalence class partitioning, boundary value analysis, decision tables, random testing, state transitions, and pairwise testing reveal externally observable defects. White-box methods such as control-flow graphs, cyclomatic complexity, branch coverage, data-flow analysis, and mutation testing provide deeper evidence when source code is available.

Selenium, JUnit, JaCoCo, Postman, JMeter, pytest, and Coverage.py support repeatable automation and objective measurement. However, a tool report is only one part of the quality argument. Effective test adequacy requires traceability to approved requirements, representative and boundary-focused test data, reproducible defect reports, regression testing, and explicit evaluation of residual risks.